

$$v = \text{self}.x[3]$$

$$\theta = \text{self}.x[2]$$

PAGE
DATE

$$\begin{aligned} \cdot \dot{x} &= v \cos(\theta) = \text{self}.x.\text{dot}[0] \\ \cdot \dot{y} &= v \sin(\theta) = \text{self}.x.\text{dot}[1] \\ \cdot \dot{\theta} &= \frac{v \tan(\delta)}{L} = \text{self}.x.\text{dot}[2] \end{aligned}$$

$$\cdot \dot{v} = (k_a u_{\text{throttle}}) - (C_{\text{drag}} v^2) - C_{\text{roll}}$$

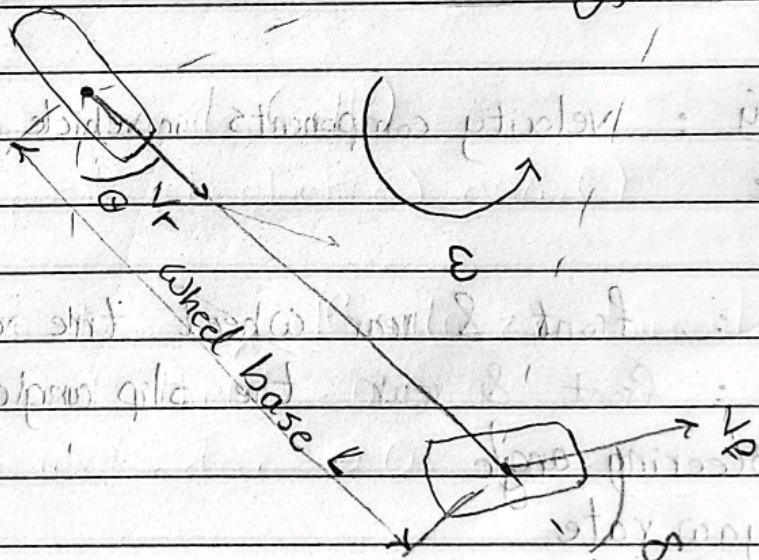
self.k_a = powertrain acceleration gain

self.C_drag = aerodynamic drag coefficient

self.C_roll = rolling resistance coefficient

"how much force resists the motion of tires compared to the weight pressing it into the surface."

$$\cdot \dot{v} = a = (\text{self}.k_a * \text{self}.u[0]) - (\text{self}.C_{\text{drag}} * v^2) - (\text{self}.C_{\text{roll}} * \text{np.sign}(v))$$



Forward Euler integration:

Updating car's state

new state = old state + (rate of change * dt)

(ex) $\text{self.x}[0] = \text{self.x}[0] + (\text{self.x_dot}[0] * dt)$

Speed Clamping:

`np.clip()` Function from the NumPy library acts as the limit to not let the speed exceed ~~self.speed~~ `self.max_speed` value.

$\text{self.x}[3] = \text{float}[\text{np.clip}(\text{self.x}[3], -\text{self.max_speed}, \text{self.max_speed})]$

Heading normalization:

It's the process of converting angle or orientation measurement into a standard, bounded range $[-180^\circ, 180^\circ]$

to prevent wrapping errors during navigation

while $\text{self.x}[2] > \text{math.pi}$:

$\text{self.x}[2] -= 2 * \text{math.pi}$

while $\text{self.x}[2] < -\text{math.pi}$:

$\text{self.x}[2] += 2 * \text{math.pi}$

```
def cmd_callback(self, msg: Twist):  
    raw_throttle = float(msg.linear.x) / self.max_linear_vel  
    raw_steer = (float(msg.angular.z) / self.max_angular_vel) * self.max_steer_rad
```

```
(11) self.last_cmd_time = self.get_clock().now()  
    elapsed_time =  
    (now - self.last_cmd_time).nanoseconds / 1e9
```

A ROS2 duration object holds its value in nanoseconds.
So we divide by $1e9$ to get the difference in seconds.

```
def publish_commands(self):  
    now = self.get_clock().now()
```

```
    if elapsed_time > self.auto_zero_timeout:
```

```
        self.current_throttle = 0.0
```

```
        self.current_steer = 0.0
```

```
    throttle_msg = Float32()
```

```
    throttle_msg.data = self.current_throttle
```

```
    self.throttle_pub.publish(throttle_msg)
```

Implementation of PID Controller

error = target vel - current vel

P_term = self.Kp * error

self.integral += error * self.dt

self.integral = np.clip(self.integral, -self.integral_limit, self.integral_limit) "Anti windup"

* Clamps the the value of the integral term

I_term = self.Ki * self.integral

derivative = (error - self.prev_error) / self.dt

D_term = self.Kd * derivative

Update previous error for the next loop

self.prev_error = error

Control_output = P_term + I_term + D_term

Clamped_output = np.clip(Control_output, -self.max_brake, self.max_throttle)

return float(output)

Velocity profiler

The only force keeping the vehicle on its curved path is the friction between the tires & the surface. To prevent sliding the required centripetal acceleration $\frac{v^2}{R}$ can't exceed the max acceleration provided by friction ($\mu \cdot g$)

$$\frac{v^2}{R} = \mu g \quad v = \sqrt{\mu g R}$$

$$v = \sqrt{\frac{a_{max}}{\kappa}}$$

$$\kappa (\text{kappa}) = \frac{1}{R}$$

ex. A sharp hairpin turn creates a tiny circle, small R , high curvature (κ)

Lateral PID

PAGE
DATE

cte: cross track error

$$\text{Heading error} = (\psi_{\text{vehicle}} - \psi_{\text{path}})$$

ψ_{vehicle} : exact direction the car is ~~direct~~ pointing.

ψ_{path} : exact direction the road itself is heading.

$$\text{pid_cte} = \text{P_term} + \text{I_term} + \text{D_term}$$

$$\text{Yaw term} = k_{\text{yaw}} * \text{heading_err}$$

Pure Pursuit Model

PAGE
DATE

$$\text{lookahead distance} = \text{velocity gain} \cdot \text{velocity} + \text{min lookahead distance}$$

As the velocity increases the equation function increases the lookahead distance.

At low velocities (during taking sharp corners) the lookahead distance should be small to avoid corner cutting.

$$l_d = \text{self.kv} * v + \text{self.l_min}$$

↳ to make sure that l_d is always greater than the wheelbase length

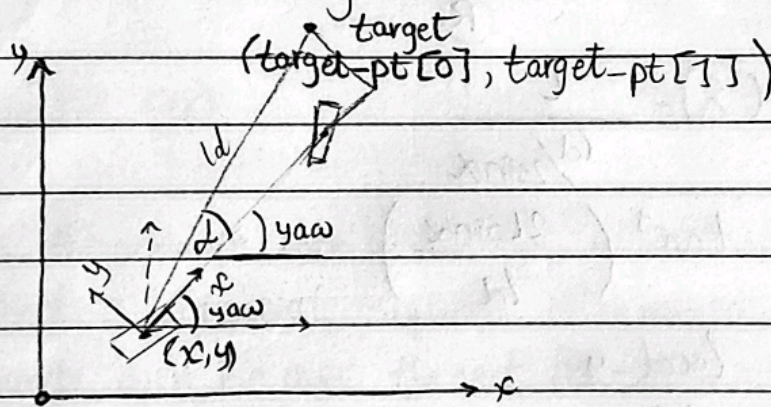
* Finding target way point:

- ① we calculate distance between the car & every path point
- ② Determine the index of closest point & total number of track points.

③ compare the distance to every path point with the lookahead distance. If distance $>$ lookahead then set this point as the new target point.

* Computing steering to follow path points :

① Transfer the target to vehicle's local frame.



Transition of origin

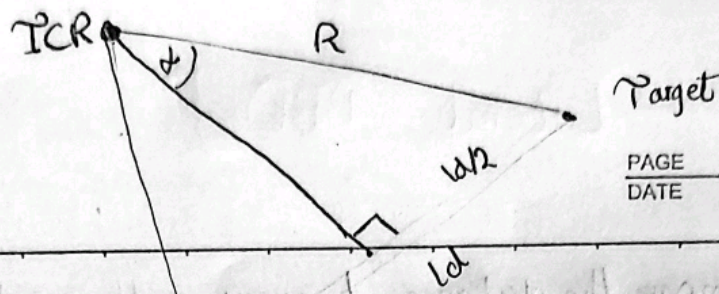
$$\Delta x = \text{local } x = \text{target_pt}[0] \cos(\theta) - \text{target_pt}[1] \sin(\theta)$$

$$\Delta y = \text{local } y = \text{target_pt}[0] \sin(\theta) + \text{target_pt}[1] \cos(\theta)$$

Rotation of axes

$$\text{local } x = \Delta x \cos(\theta) + \Delta y \sin(\theta)$$

$$\text{local } y = -\Delta x \sin(\theta) + \Delta y \cos(\theta)$$



$$\sin \alpha = \frac{l/2}{R}$$

$$R = \frac{l}{2 \sin \alpha}$$

relation between (δ) , (L) , (R)

$$\tan(\delta) = \frac{L}{R}$$

$$\tan(\delta) = \frac{L}{l/2 \sin \alpha}$$

$$\delta = \tan^{-1} \left(\frac{2L \sin \alpha}{l} \right)$$

$$\sin \alpha = \frac{\text{local-}y}{l}$$

$$\delta = \tan^{-1} \left(\frac{2L \cdot \text{local-}y}{l^2} \right)$$

MPC

PAGE _____
DATE _____

① Horizon and Bounds

We calculate a sequence of steps into the future. This lookahead horizon is called prediction horizon; that is what makes it a predictive not a reactive controller.

- Determine effective horizon N
- Construct variable bounds for decision vector u

② Objective (u)

This is the absolute core of the MPC algorithm. It determines how bad a potential sequence of steering and throttle commands will be over the next few seconds, until it finds the lowest possible cost.

③ Numerical optimization & control extraction

We choose the option with minimised costs to be the optimal solution, and then we extract control inputs: ① throttle ② steering